

Esercizio 2 – Esplorazione di Processi, Thread, Handle e Registro di Windows

Studente	Alessandro
Corso	Cyber Security & Ethical Hacking – Cisco CyberOps
Laboratorio	Esercizio 2: Esplorazione di Processi, Thread, Handle e Registro di Windows
Strumento	Sysinternals Process Explorer / Editor del Registro di Windows
Data	15/06/2026

In questo report documento lo svolgimento dell'Esercizio 2, dedicato all'esplorazione di processi, thread e handle tramite **Process Explorer** della Suite Sysinternals, e alla modifica di una chiave del Registro di Windows. Per ciascuna fase riporto le operazioni svolte, gli screenshot raccolti e le risposte alle domande presenti nella traccia.

Obiettivi del laboratorio

L'obiettivo dell'esercitazione è esplorare i processi, i thread e gli handle attivi su un sistema Windows utilizzando Process Explorer, e successivamente modificare un'impostazione tramite il Registro di Windows. Il lavoro è suddiviso in tre parti:

- Parte 1 – Esplorazione dei Processi
- Parte 2 – Esplorazione di Thread e Handle
- Parte 3 – Esplorazione del Registro di Windows

Ambiente di lavoro

Ho svolto l'esercizio su una macchina virtuale Windows (host visualizzato come **THETAITonyStark**) con accesso a internet, utilizzando Process Explorer (*procexp64a.exe*) della Suite Sysinternals e l'Editor del Registro di sistema (*regedit*).

Parte 1 – Esplorazione dei Processi

Passo 1 – Download di Sysinternals Suite

Come primo passo ho scaricato la Sysinternals Suite dal sito ufficiale Microsoft (learn.microsoft.com/sysinternals/downloads/sysinternals-suite) ed ho estratto tutti i file dall'archivio in una cartella locale, lasciando il browser aperto per i passaggi successivi.

Passo 2 – Esplorazione di un processo attivo

Ho avviato *procexp64a.exe* (Process Explorer) ed ho accettato l'Accordo di Licenza con l'Utente Finale (EULA) quando richiesto. Si è quindi aperta la finestra principale, che mostra l'elenco completo dei processi attivi organizzati in una struttura ad albero, con le colonne CPU, Private Bytes, Working Set, PID, Description e Company Name.

Per individuare il processo del browser web ho utilizzato l'icona **Find Window's Process** (il mirino) trascinandola sulla finestra di Microsoft Edge: Process Explorer ha evidenziato automaticamente in blu il processo **msedge.exe** (PID 4408) all'interno dell'albero, mostrando anche tutti i processi figli msedge.exe relativi alle varie schede e componenti del browser (renderer, GPU, estensioni, ecc.). Nella stessa finestra si possono notare altri esempi di gerarchia genitore/figlio analoghi: **WindowsTerminal.exe**, che genera i processi figli *OpenConsole.exe* e *powershell.exe*, e **MicrosoftEdgeUpdate.exe**, che genera il processo *MicrosoftEdge_ARM64...setup.exe* per l'aggiornamento del browser.

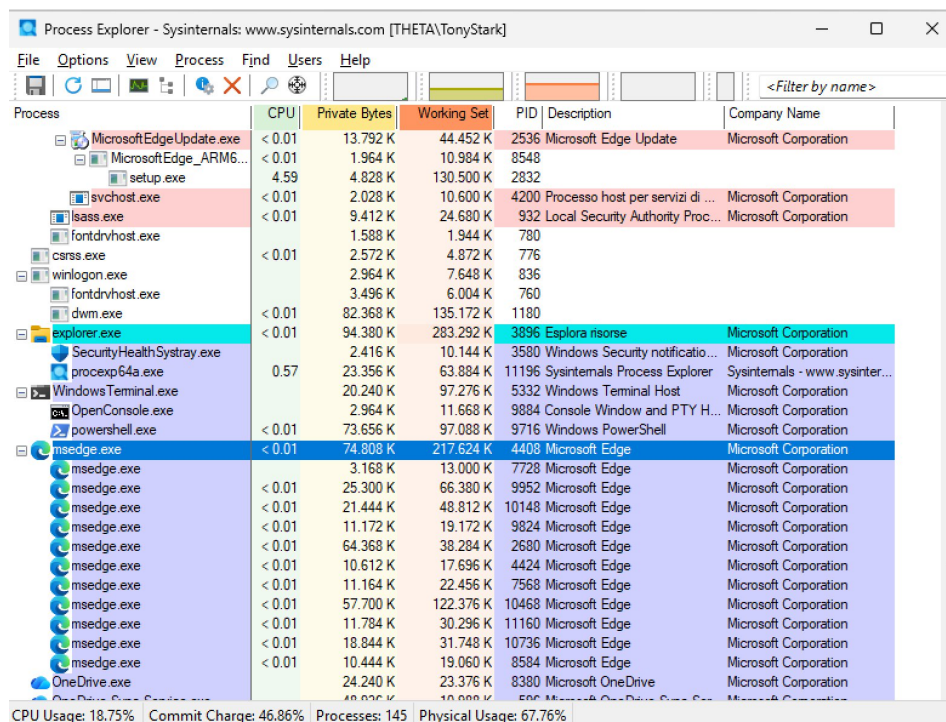


Figura 1 – Process Explorer: il processo msedge.exe (PID 4408) evidenziato in blu dopo l'utilizzo dell'icona Find Window's Process sulla finestra di Microsoft Edge. Sono visibili anche gli esempi di gerarchia WindowsTerminal.exe → OpenConsole.exe/powershell.exe e MicrosoftEdgeUpdate.exe → MicrosoftEdge_ARM64...setup.exe.

Ho quindi fatto clic con il pulsante destro sul processo msedge.exe selezionato e scelto **Kill Process**, confermando con OK.

Domanda: Cosa è successo alla finestra del browser web quando il processo è stato terminato?

Risposta: Nel momento in cui ho terminato il processo msedge.exe principale, la finestra del browser si è chiusa immediatamente e in modo brusco, senza alcuna richiesta di conferma o possibilità di salvare lo stato delle schede aperte (a differenza di una chiusura "normale" tramite la X della finestra). In Process Explorer, tutti i processi figli msedge.exe collegati (renderer delle schede, GPU process, estensioni) sono scomparsi dall'elenco insieme al processo principale, poiché in un'architettura multi-processo come quella di Edge/Chromium il processo principale (browser process) coordina e mantiene in vita i processi figli: terminandolo, anche tutti i suoi figli vengono terminati a cascata.

Passo 3 – Avviare un altro processo

Ho aperto un Prompt dei Comandi (Start > cercato "Prompt dei comandi"). Trascinando nuovamente l'icona Find Window's Process sulla finestra del prompt, Process Explorer ha evidenziato il processo **cmd.exe** (PID 8800). Osservando l'albero dei processi (Figura 2) si nota che cmd.exe è figlio diretto di **explorer.exe** (il suo processo genitore), e a sua volta cmd.exe ha come processo figlio **conhost.exe** (PID 8956), che fornisce l'interfaccia host della console al Prompt dei Comandi.

Sono poi tornato sulla finestra del Prompt dei Comandi ed ho lanciato un comando **ping**.

Domanda: Cosa è successo durante il processo ping?

Risposta: Durante l'esecuzione del comando ping, in Process Explorer è apparso temporaneamente un nuovo processo figlio di cmd.exe (PING.EXE), visibile per la durata dell'invio e della ricezione dei pacchetti ICMP. Contestualmente ho notato un piccolo picco di utilizzo della CPU e di attività di rete associato al processo, che durante l'esecuzione del comando passa rapidamente dallo stato di attesa a quello in esecuzione. Una volta completato il ping (terminati gli echo request/reply e stampato il riepilogo statistico a schermo), il processo PING.EXE è terminato automaticamente e scomparso dall'elenco, lasciando nuovamente cmd.exe come unico processo attivo in quel ramo dell'albero.

Esaminando l'elenco dei processi attivi, ho notato il processo figlio **conhost.exe** (PID 8956) e, per verificarne l'affidabilità, ho fatto clic con il pulsante destro su di esso selezionando **Check VirusTotal**. Ho accettato i Termini di Servizio di VirusTotal (Yes) e confermato il messaggio successivo (OK). Espandendo la finestra verso destra fino a visualizzare la colonna **VirusTotal**, è apparso il link **0/78** in corrispondenza di conhost.exe (Figura 2): cliccandolo, il browser predefinito si è aperto mostrando la pagina dei risultati di VirusTotal, dalla quale risulta che **0 motori antivirus su 78** hanno rilevato contenuti malevoli nel file conhost.exe, confermando quindi che si tratta del legittimo file di sistema Microsoft e non di un eseguibile sospetto.

Process	CPU	Private Bytes	Working Set	PID	Description	Company Name	VirusTotal
svchost.exe		4.120 K	15.520 K	4536	Processo host per servizi di ...	Microsoft Corporation	
svchost.exe		1.976 K	9.948 K	7156	Processo host per servizi di ...	Microsoft Corporation	
svchost.exe		7.824 K	35.288 K	3212	Processo host per servizi di ...	Microsoft Corporation	
lsass.exe		9.564 K	17.224 K	932	Local Security Authority Proc...	Microsoft Corporation	
fontdrvhost.exe		1.588 K	1.684 K	780			
csrss.exe	0.19	2.596 K	3.124 K	776			
winlogon.exe		3.256 K	13.248 K	836			
fontdrvhost.exe		3.568 K	2.456 K	760			
dwim.exe	< 0.01	90.280 K	108.232 K	1180			
explorer.exe	< 0.01	104.528 K	159.904 K	3896	Esplora risorse	Microsoft Corporation	
SecurityHealthSystray.exe		2.472 K	6.748 K	3580	Windows Security notificatio...	Microsoft Corporation	
procexp64a.exe	0.19	29.576 K	40.844 K	11196	Sysinternals Process Explorer	Sysinternals - www.sysinter...	
msedge.exe	< 0.01	68.464 K	79.296 K	10452	Microsoft Edge	Microsoft Corporation	
msedge.exe		3.072 K	3.780 K	4408	Microsoft Edge	Microsoft Corporation	
msedge.exe		22.652 K	33.388 K	4160	Microsoft Edge	Microsoft Corporation	
msedge.exe		18.424 K	22.744 K	10404	Microsoft Edge	Microsoft Corporation	
msedge.exe		11.088 K	4.680 K	9832	Microsoft Edge	Microsoft Corporation	
msedge.exe		47.528 K	9.612 K	11024	Microsoft Edge	Microsoft Corporation	
msedge.exe		85.348 K	91.764 K	6036	Microsoft Edge	Microsoft Corporation	
msedge.exe		18.924 K	6.108 K	2804	Microsoft Edge	Microsoft Corporation	
msedge.exe		11.652 K	15.316 K	10624	Microsoft Edge	Microsoft Corporation	
msedge.exe		10.960 K	6.248 K	9516	Microsoft Edge	Microsoft Corporation	
cmd.exe		2.260 K	2.032 K	8800	Processore dei comandi di ...	Microsoft Corporation	
conhost.exe		1.764 K	4.212 K	8956	Host finestra console	Microsoft Corporation	0/78
regedit.exe		5.572 K	25.948 K	8832	Editor del Registro di sistema	Microsoft Corporation	
OneDrive.exe		24.416 K	9.444 K	8380	Microsoft OneDrive	Microsoft Corporation	
OneDrive.Sync.Service.exe		48.588 K	7.264 K	596	Microsoft OneDrive Sync Ser...	Microsoft Corporation	

CPU Usage: 0.57% Commit Charge: 51.36% Processes: 141 Physical Usage: 59.47%

Figura 2 – Process Explorer: albero dei processi con cmd.exe (PID 8800) e il suo processo figlio conhost.exe (PID 8956), con il risultato della scansione VirusTotal (0/78) visibile nella colonna VirusTotal. È visibile anche regedit.exe (PID 8832), aperto per la Parte 3 dell'esercizio.

Infine ho fatto clic con il pulsante destro sul processo cmd.exe e selezionato **Kill Process**.

Domanda: Cosa è successo al processo figlio conhost.exe?

Risposta: Terminando il processo padre cmd.exe, anche il processo figlio conhost.exe è stato terminato automaticamente ed è scomparso dall'elenco di Process Explorer. Questo conferma la relazione di dipendenza tra processo genitore e processo figlio: conhost.exe esiste esclusivamente per fornire l'host della finestra di console al processo cmd.exe, quindi quando quest'ultimo viene chiuso (o terminato a forza), anche il suo host di console viene chiuso, non avendo più motivo di esistere. Anche la finestra del Prompt dei Comandi, di conseguenza, si è chiusa immediatamente.

Parte 2 – Esplorazione di Thread e Handle

In questa parte ho esplorato i thread e gli handle del processo conhost.exe, utilizzando sempre Process Explorer. Un **thread** è un'unità di esecuzione all'interno di un processo, mentre un **handle** è un riferimento astratto a un blocco di memoria o a un oggetto gestito dal sistema operativo.

Passo 1 – Esplorare i thread

Dopo aver aperto un nuovo Prompt dei Comandi, in Process Explorer ho fatto clic con il pulsante destro su **conhost.exe** (PID 8956) e selezionato **Properties...** Nella finestra delle proprietà ho aperto la scheda **Threads**, che mostra i thread attivi del processo.

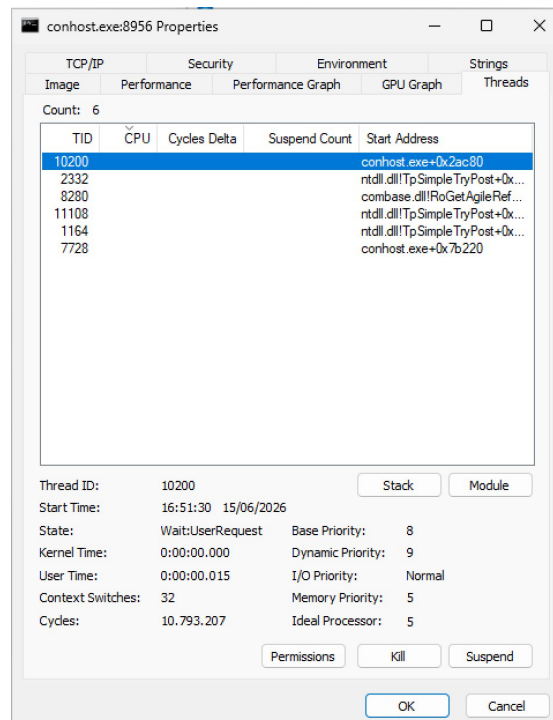


Figura 3 – Finestra Proprietà di conhost.exe (PID 8956), scheda Threads: elenco dei 6 thread attivi e dettaglio del thread selezionato (TID 10200).

Domanda: Che tipo di informazioni sono disponibili nella finestra Proprietà?

Risposta: Nella scheda Threads della finestra Proprietà sono disponibili, per ciascun thread del processo (nel mio caso 6 thread in totale): il **TID** (Thread ID), la percentuale di **CPU** utilizzata, il **Cycles Delta**, il **Suspend Count** e lo **Start Address**, cioè la funzione/modulo da cui il thread ha avuto inizio (ad esempio conhost.exe+0x2ac80, ntdll.dll!TpSimpleTryPost o combase.dll!RoGetAgileReference). Selezionando un singolo thread (nel mio caso il TID 10200), vengono mostrate ulteriori informazioni di dettaglio: **Thread ID**, **Start Time** (data e ora di avvio del thread, 16:51:30 del 15/06/2026), **State** (Wait:UserRequest, cioè in attesa di un'azione dell'utente), **Kernel Time** e **User Time** (tempo di CPU trascorso rispettivamente in modalità kernel e utente), **Context Switches** (32, il numero di cambi di contesto), **Cycles** (10.793.207 cicli di CPU consumati), **Base Priority** (8) e **Dynamic Priority** (9), **I/O Priority** (Normal), **Memory Priority** (5) e **Ideal Processor** (5, il core di CPU su cui il thread dovrebbe preferibilmente essere eseguito). Sono infine disponibili i pulsanti **Stack** (per vedere lo stack di chiamate), **Module**, **Permissions**, **Kill** e **Suspend**, che permettono rispettivamente di analizzare o intervenire direttamente sul singolo thread.

Passo 2 – Esplorare gli handle

Tornato alla finestra principale di Process Explorer, ho selezionato **View > Lower Pane View > Handles** per visualizzare nel riquadro inferiore l'elenco degli handle associati al processo selezionato.

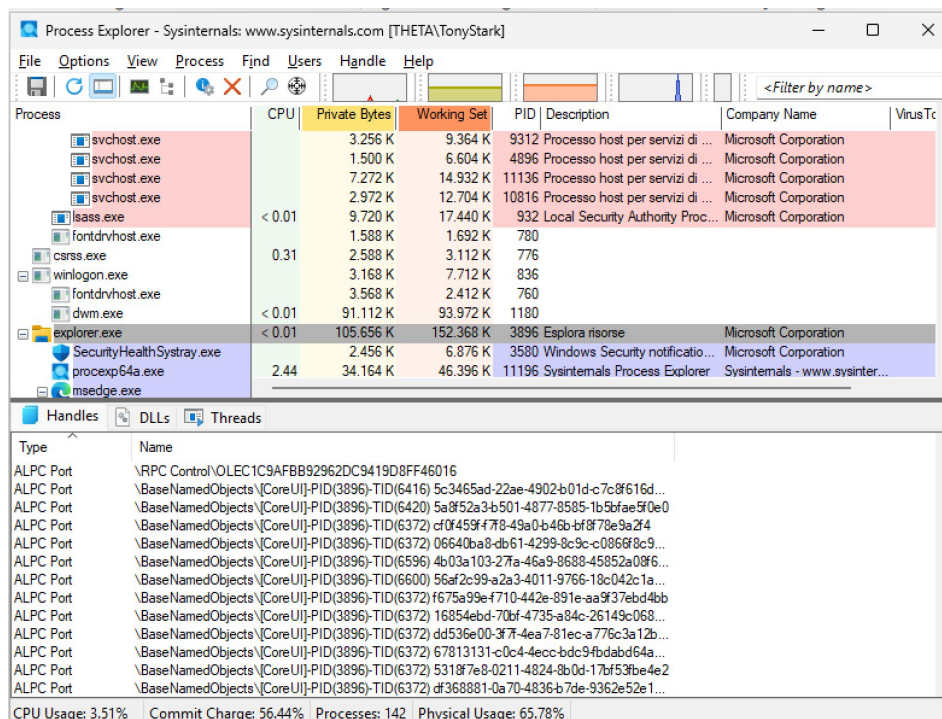


Figura 4 – Riquadro inferiore "Handles": elenco degli handle di tipo ALPC Port e RPC Control associati al processo selezionato (explorer.exe, PID 3896).

Domanda: Esaminare gli handle. A cosa puntano gli handle?

Risposta: Gli handle elencati nel riquadro inferiore sono riferimenti che il processo selezionato mantiene aperti verso oggetti gestiti dal kernel di Windows (file, chiavi di registro, eventi, sezioni di memoria, thread, porte di comunicazione, ecc.). Nella schermata che ho catturato, la quasi totalità degli handle è di tipo **ALPC Port** (Advanced Local Procedure Call): si tratta di porte di comunicazione inter-processo identificate da nomi del tipo `\BaseNamedObjects\{CoreUI}-PID(3896)-TID(XXXX)-{GUID}`, utilizzate dal processo per scambiare messaggi con altri thread/componenti del sistema (in questo caso con i thread dell'interfaccia CoreUI di explorer.exe). È presente anche un handle di tipo `\RPC Control\{GUID}`, usato per le chiamate Remote Procedure Call. In generale, quindi, gli handle non contengono direttamente i dati, ma **puntano** a oggetti di sistema condivisi, permettendo al processo di interagire con essi senza doverne conoscere la posizione fisica in memoria.

Parte 3 – Esplorazione del Registro di Windows

Il Registro di Windows è un database gerarchico che memorizza la maggior parte delle impostazioni di configurazione del sistema operativo e dell'ambiente desktop. È organizzato in cinque hive principali: **HKEY_CLASSES_ROOT** (associazioni file, ProgID, CLSID, IID), **HKEY_CURRENT_USER** (impostazioni dell'utente attualmente connesso), **HKEY_LOCAL_MACHINE** (configurazione del computer locale), **HKEY_USERS** (impostazioni di tutti gli utenti, di cui HKEY_CURRENT_USER è una sottochiave) e **HKEY_CURRENT_CONFIG** (informazioni hardware utilizzate all'avvio).

Ho aperto l'Editor del Registro di sistema (Start > "regedit" > Sì alla richiesta UAC) e ho navigato fino alla chiave **HKEY_CURRENT_USER \ Software \ Sysinternals \ Process Explorer**, dove ho individuato il valore **EulaAccepted**, precedentemente impostato a 0x00000001 (1) in seguito all'accettazione dell'EULA di Process Explorer effettuata nei passi precedenti.

Ho quindi fatto doppio clic sul valore EulaAccepted, aprendo la finestra "Modifica valore DWORD (32 bit)", ed ho modificato il **Dati valore** da 1 a 0 (con Base impostata su Esadecimale), per indicare che l'EULA non risulta più accettato. Ho confermato con OK.

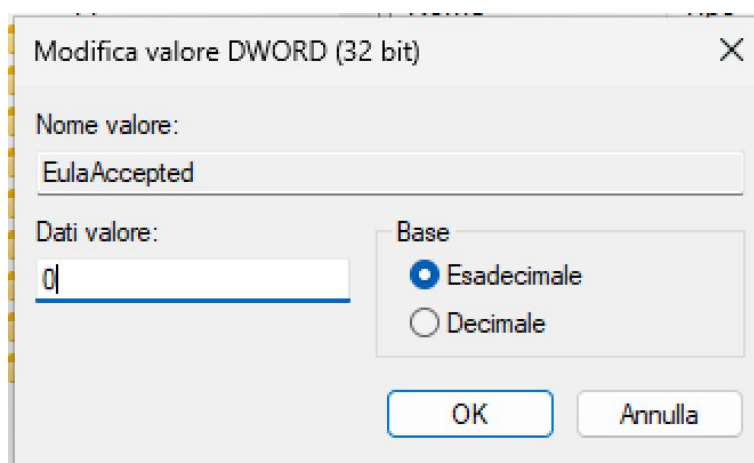


Figura 5 – Finestra "Modifica valore DWORD (32 bit)" per la chiave EulaAccepted, con il dato valore impostato a 0 in base esadecimale.

Domanda: Qual è il valore per questa chiave di registro nella colonna Dati (Data)?

Risposta: Dopo la modifica, nella colonna Dati (Data) dell'Editor del Registro la chiave EulaAccepted mostra il valore **0x00000000 (0)**, in formato esadecimale a 32 bit. Il valore 0 indica che, dal punto di vista dell'applicazione, l'EULA di Process Explorer non è (più) stato accettato dall'utente.

Per verificare l'effetto della modifica, ho navigato alla cartella SysInternalsSuite e ho riavviato Process Explorer (procxp.exe).

Domanda: Quando apri Process Explorer, cosa vedi?

Risposta: All'apertura di Process Explorer, prima ancora che venga mostrata la finestra principale con l'elenco dei processi, è apparsa nuovamente la finestra dell'**EULA (End User License Agreement)**, che richiede di accettare i termini di licenza prima di poter procedere. Questo accade perché, ad ogni avvio, l'applicazione controlla il valore della chiave di registro EulaAccepted: avendolo impostato a 0, Process Explorer non trova più traccia di un'accettazione precedente e ripropone quindi la richiesta di accettazione dell'accordo di licenza, esattamente come avviene al primissimo avvio dell'applicazione.

Conclusioni

Attraverso questa esercitazione ho approfondito l'uso di **Process Explorer** come strumento di analisi avanzata dei processi in esecuzione su un sistema Windows, osservando in modo concreto i concetti di processo genitore/figlio (explorer.exe → cmd.exe → conhost.exe, WindowsTerminal.exe → OpenConsole.exe/powershell.exe), gli effetti della terminazione forzata di un processo sui suoi discendenti, e l'utilità dell'integrazione con VirusTotal per una rapida verifica di affidabilità di un eseguibile sospetto.

Ho inoltre analizzato i **thread** e gli **handle** di un processo, comprendendo come ogni processo sia in realtà composto da una o più unità di esecuzione (thread) e mantenga riferimenti (handle) verso oggetti di sistema come porte ALPC e canali RPC, fondamentali per la comunicazione inter-processo.

Infine, modificando la chiave di registro **EulaAccepted** in HKEY_CURRENT_USER ho potuto verificare concretamente come un'applicazione possa memorizzare lo stato di accettazione di un EULA nel Registro di Windows, e come la manomissione di tale valore (da 1 a 0) ne forzi nuovamente la richiesta all'avvio successivo. Questo aspetto è particolarmente interessante dal punto di vista della sicurezza, in quanto evidenzia come configurazioni e stati applicativi "persistenti" siano spesso memorizzati in chiari valori del Registro, potenzialmente leggibili e modificabili da chiunque abbia accesso al profilo utente.